

The Three-Plane Compute Fabric: Consensus-Managed, ZAP-Transported Decentralized AI Inference

Hanzo AI — ML Systems

Architecture Blueprint — *One Source, Every Backend* series, 2026

Abstract

A heterogeneous GPU fleet already runs a single model across ROCm, CUDA, and Metal today: the Hanzo engine’s socket-ring shards a model tensor- and pipeline-parallel across mismatched accelerators over a canonical `f32` wire, and a `hanzo cluster` command onboards a worker by scanning a QR token. But that ring is configured by a static JSON file handed out of band, and its bytes cross the wire as *plaintext TCP* between nodes that never authenticate each other. It is a LAN appliance, not a network. This paper is the blueprint for the missing two-thirds: turning that ring into a permissionless, byzantine-fault-tolerant AI inference *network* that any developer — or any public-cloud GPU miner — can join and be paid for useful work. The thesis is one of *decomplection*: a serving cluster has three concerns that monolithic stacks braid together and we hold apart as three orthogonal, independently-complete planes. The **control plane** is a Lux *Quasar* consensus chain: membership, permissionless miner admission, layer placement, and config distribution become BFT-agreed state, with the Photon committee-selection primitive (Fisher–Yates over a luminance-weighted population) reused to sample GPU nodes and luminance re-bond from consensus-vote success to *validated inference contribution*. The **transport** is *ZAP*, the Lux zero-copy wire, whose post-quantum handshake (*WrapPQ* over an authenticated *Session* exposing a verified 32-byte *PeerID*) carries *both* consensus control messages *and* tensor activations over one PQ-authenticated channel — retiring the plaintext ring sockets and converting the QR join token from a bearer secret into a handshake credential. The **data plane** is the existing inference ring, unchanged in kind: it receives its topology from consensus instead of a JSON file and moves its tensors over ZAP instead of raw TCP. We are scrupulous about the built/designed line. Built and cited to source: the cross-vendor ring, the `hanzo cluster` scan-to-join CLI, the ZAP Go wire with its PQ handshake, the Quasar engine with Photon/Wave/Focus and BLS+Pulsar+ML-DSA threshold signing, and the ZAP measurements (17× serialization and 345× parse versus JSON-RPC, ~4.5× versus gRPC with zero marshal/unmarshal). Designed, and labelled as such: consensus-driven membership, the ZAP-swapped ring transport, miner admission, and result verification by threshold-signing over inference outputs. No benchmark here is invented; every measured number is attributed to the code that produced it. The contribution is the architecture and its honest status, offered as a blueprint the wider AI-infrastructure community can build on.

1 Introduction

The dominant shape of a model-serving stack is a monolith: one operator, one datacenter, one process tree that owns scheduling, transport, and tensor math all at once. vLLM, TGI, and Triton each fuse “who is in the cluster,” “how bytes move between ranks,” and “what the GPUs compute” into a single trusted binary on a private network. That fusion is fine when the operator owns every

GPU. It fails the moment the fleet is *heterogeneous* (a ROCm mini-PC, a rented CUDA instance, an Apple laptop), *permissionless* (a stranger’s GPU wants to contribute and be paid), or *untrusted* (the node holding your activations is not yours). None of those three failures is a math problem; each is a *concerns* problem. The cluster braids together things that want to be separate.

This paper takes the opposite stance, the one our engineering culture states as a law: one and only one way to do everything, composable and orthogonal. A decentralized inference cluster has exactly three concerns, and we give each its own plane:

- **Control** — who is in the cluster, who may join, which node holds which layers, and what the run configuration is. This is agreement among mutually distrusting parties: a *consensus* problem.
- **Transport** — how a byte gets from one node to another, and how each end knows who the other is. This is an *authenticated wire* problem.
- **Data** — the actual tensor exchange: activations forward, gradients or all-reduce sideways. This is a *numerics and topology* problem.

The claim is that these three are genuinely orthogonal, that Hanzo and Lux already ship a best-in-class primitive for each, and that composing them — rather than reinventing a monolith — yields something no monolith can be: a GPU network anyone can join. The control primitive is Lux *Quasar* consensus [4, 5]; the transport is *ZAP* [6, 7]; the data plane is the Hanzo cross-backend inference engine [1].

The honesty frame. This is an architecture paper, and its credibility rests on not blurring what runs today with what is designed. We adopt three labels, used inline throughout and collected in the status table of §8: **[built]** for code on **main**, cited to its file; **[partial]** for a primitive that is compiled and exercised but not yet wired end-to-end; and **[design]** for a proposed composition. The measured numbers we quote (ring parity across vendors, *ZAP* throughput and latency, Photon and luminance costs) are all **[built]**, each attributed to the benchmark that produced it. The network-level assembly — consensus as the live scheduler, the ring running over *ZAP*, miner admission and payment — is **[design]**, and never dressed as more.

Contribution. We (i) inventory the built substrate with source citations (§2); (ii) state the three-plane architecture and prove its concerns are separable (§3); (iii) give the threat model and a result-integrity scheme for byzantine miners (§4); (iv) bind the economic layer to the existing luminance primitive (§5); (v) lay out a staged migration from the static-config LAN cluster to a public network (§6); and (vi) publish an honest built-versus-designed status table (§8) and the limitations (§9).

2 The substrate that exists today

Before proposing a network we account for what is already on **main**. Every claim in this section is **[built]** and cites the file that implements it. The point of the inventory is that the three planes are not vaporware waiting to be written: each has a mature primitive, and the work is composition, not invention.

2.1 Data plane: the heterogeneous inference ring

The Hanzo engine already runs one model split across several GPUs of *different vendors*. Its `distributed` module (`hanzo-engine/src/distributed.rs`, `hanzo-quant/src/distributed/`) provides two backends behind one `Comm` abstraction: NCCL for homogeneous NVIDIA, and a portable *ring* backend for everything else. The ring does tensor-parallel and pipeline-parallel sharding by exchanging activations with its right-hand neighbor, rank r dialing rank $(r+1) \bmod N$ in an all-reduce cycle (`hanzo-quant/src/distributed/socket.rs`). Cross-vendor generation works because the ring wire is a *canonical f32* format and reductions take a backend-independent CPU-f32 path with a canonical tie-break (`hanzo-engine/src/ops.rs`), so a ROCm box and a CUDA box agree bit-for-bit on the reduced tensor rather than each rank reinterpreting the other's raw 16-bit words in the wrong `f16-versus-bf16` layout. The topology is a static struct read from a JSON file named by the `RING_CONFIG` environment variable (Listing 1): master address, per-rank ports, this rank's index, and the world size. The cross-backend engine underneath — five backends, the full GGUF quant zoo, decode parity with `llama.cpp` — is the subject of the companion stack paper [1]; here it is simply the data plane.

Listing 1: The data-plane topology today: a static `RING_CONFIG` JSON. Every node needs the full member list up front; nothing authenticates a peer.

```
{ "master_ip": "10.0.0.1", "master_port": 8000,  
  "port": 8001, "right_ip": "10.0.0.2", "right_port": 8001,  
  "rank": 0, "world_size": 4 }
```

Two properties of this substrate matter for everything below. First, it is *correct across vendors* — the hard part, cross-backend numeric agreement, is solved. Second, it is *unauthenticated plaintext*: the ring exchanges raw bytes over `std::net::TcpStream` with `write_all/read_exact` and ships its control messages as `serde_json` over the same socket. It is safe only on a trusted private mesh. That is the gap the transport plane closes.

2.2 The onboarding UX: hanzo cluster scan-to-join

Handing every node a JSON file out of band does not scale past a lab. The `hanzo cluster` subcommand (`hanzo-cli/src/cluster.rs`) already replaces it with a scan-to-join flow. `hanzo cluster init` on the head (rank 0) enumerates the ring, prints a *join token* and its QR code, and launches; `hanzo cluster join` on a worker decodes the token and launches as a ring daemon. The token is a `bs58`-encoded JSON `JoinToken` (Listing 2) — a nonce, the head address, the world size, the ordered ring addresses, and the model — compact enough to fit in one QR or one paste. This is the correct *shape* of onboarding: a worker needs one string to join. But observe what the token is today: a *bearer credential in cleartext*. Anyone who photographs the QR can join the ring, and the ring cannot tell an invited worker from an intruder, because the token carries topology, not identity. Holding that observation is the whole of §4.

Listing 2: The join token (`hanzo-cli/src/cluster.rs`): `bs58(JSON)` so it fits one QR. It is topology, not identity — a bearer secret today.

```
struct JoinToken {  
    nonce: u64, // ring id, printed {:016x}  
    master: String, // rank-0 (head) reachable IP  
    world_size: usize,  
    addrs: Option<Vec<String>>, // full ordered ring [rank0..rankN-1]  
    model: String,  
}
```

2.3 Transport: ZAP, the zero-copy post-quantum wire

ZAP (Zero-copy Application Protocol, github.com/luxfi/zap) is the Lux universal wire [6, 7]. Its serialization is zero-copy: a message is read directly out of the network buffer with no parse and no allocation (measured [built] parse 2.9 ns/op at zero allocations; build 44 ns/op). Against the protocols an AI stack would otherwise reach for, the wire eliminates the marshal/unmarshal step entirely: 17× faster serialization and 345× faster parse than JSON-RPC (the MCP transport), ~157× faster than protobuf on deserialization (21 ns versus 3231 ns, zero allocations), and ~4.5× faster than gRPC with the whole codec layer deleted rather than optimized [6, 7, 5]. End-to-end it sustains 114K transactions/s on a single connection and 376K across 20 parallel connections [5].

For a public network the decisive feature is not speed but *authenticated identity*. ZAP’s PQ layer (`conn_pq.go`) wraps an established connection *after* a post-quantum handshake: `handshake.Initiator.Run(tcp)` produces a `Session`, and `WrapPQ(tcp, sess)` returns an ordinary `net.Conn` that AEAD-frames every write, zeroes its keys on close, and — critically — exposes `PeerID()` [32]byte, “the verified peer identity for callers that need to route on it (e.g. validator-set membership checks).” That one method is the hinge of the entire security argument (§4): the transport already hands the control plane a cryptographically verified identity for the node on the other end.

Inside the Hanzo engine, ZAP is a first-class module (`hanzo-engine/src/zap/`). Its message and adapter surface is compiled and exercised in-process — `ZapInferenceServer` translates ZAP `ChatRequests` into the engine’s real `Request` channel and streams `ChatChunks` back, satisfying the same `ZapInference` trait a networked client would (`server.rs`, `client.rs`). We mark this [partial], honestly: the out-of-process `zap://` transport is a documented stub that carries no `capnp` build dependency yet (“flipping on the wire is mechanical,” `transport.rs`), and a few methods (`tokenize`, `detokenize`, `re_isq`) return “not yet wired.” The wire exists in Go and is production-deployed across Lux consensus, MPC, and DEX systems; the engine-side Rust surface is built but not yet turned on over a socket.

2.4 Control: the Quasar consensus engine

Lux *Quasar* (github.com/luxfi/consensus) is a DAG-native, post-quantum BFT consensus engine [4, 5]. It is built from small, orthogonal sub-protocols, four of which this paper reuses directly:

- **Photon** (`protocol/photon`) selects a K -of- N committee each round by Fisher–Yates shuffle over the node population, drawing with a rejection-sampled uniform integer (`emitter.go`) to remove the modulo bias that a grinding adversary could exploit on committee sampling. The population is weighted by *luminance*.
- **Luminance** (`photon/luminance.go`) is a per-node reputation in “lux” units: each node starts at 100, a successful vote multiplies its brightness by 1.1 (capped at 1000), a failure by 0.9 (floored at 10), and `Brightness` normalizes to a selection weight in $[0.1, 10]$. It is a cheap, bounded, exponential reputation — a brightness update costs 72 ns [built].
- **Wave** (`protocol/wave`) runs per-round threshold voting with Fast Probabilistic Consensus; a vote round costs 3.38 μ s and a K -of- N Photon selection 3.03 μ s [built].
- **Quasar** (`protocol/quasar`) seals decisions with threshold signatures across three independent cryptographic paths — BLS12-381 (48-byte aggregate), Pulsar Ring-LWE threshold, and ML-DSA-65 (FIPS 204, rolled to a 192-byte Groth16 proof) — each independently toggleable, all three in parallel under `TripleSignRound1`. A `QuasarCert` verifies in ~2–5 ms on CPU, GPU-batchable 10–100× [5].

Control plane — Quasar consensus (BFT, post-quantum)	[design]
Photon committee over GPU nodes · luminance = useful-work weight membership · miner admission · layer placement · config · result attest	
Transport plane — ZAP PQ wire	[built] / [partial]
WrapPQ session · verified PeerID · AEAD frames one channel for BOTH control messages AND tensor activations	
Data plane — Hanzo inference ring	[built]
tensor/pipeline parallel · canonical f32 all-reduce · 5 backends topology handed down by control; bytes carried by transport	

Figure 1: Three orthogonal planes. Each is independently complete and separately citable to code; the network is their composition, not a new monolith. Labels mark built versus designed per §8.

Quasar’s own inter-node transport is already ZAP, not gRPC or libp2p (`consensus/LLM.md`). Control and transport are, in the Lux stack, *already* composed; what is missing is binding that composition to the *GPU* population instead of the validator population.

3 The three-plane architecture

The blueprint is a single sentence: *run the inference ring’s topology as Quasar consensus state, and move every byte — control and tensor alike — over ZAP*. Figure 1 is the whole system. What follows gives each plane its interface and states, as a proposition, why the three do not leak into each other.

3.1 Control plane: consensus as the scheduler

The control plane replaces the static `RING_CONFIG` JSON (Listing 1) and the out-of-band QR token with *agreed state on a Quasar chain*. The chain’s VM state *is* the cluster’s control surface [design]:

- **Membership** — the set of GPU nodes, each identified by its ZAP `PeerID` (an ML-DSA public key), with advertised capability (backend, VRAM, resident models, measured decode/prefill throughput). Joining and leaving are transactions, so the member list is BFT-agreed rather than hand-distributed.
- **Miner admission** — a permissionless join. A public-cloud GPU submits a join transaction carrying its `PeerID` and a stake bond; admission is a consensus decision, not an operator’s SSH session. This is the step that turns “my four boxes” into “anyone’s GPU” (§5).
- **Placement / routing** — which node holds which contiguous layer range of which model. Pipeline-parallel assignment (node *A* holds layers 0–7, *B* holds 8–15) and the ring order become an on-chain placement map. A request is routed by reading that map, not by trusting a JSON file.
- **Config distribution** — model id, quantization, sampling defaults, ring `world_size`: the fields of `JoinToken` and `RingConfig` become chain state, so every node reads an identical, agreed configuration.

The mechanism is not new code so much as a *re-population* of Photon. Photon selects a committee by weighted Fisher–Yates over a node set; point it at the *GPU* population and it selects a

committee of GPU nodes — to ratify a placement, to admit a miner, or to verify a result (§4). Luminance, today a function of consensus-vote success, is re-bound to *validated inference contribution*: a node whose outputs pass verification brightens and is preferentially selected and rewarded; a node that stalls or returns wrong tensors dims and is sampled out. The primitive is unchanged — an exponential, bounded, cheap reputation — only the event that drives `Illuminate(id, success)` moves from “voted with the quorum” to “did correct inference work.” Crucially, *writes to consensus are rare and slow-changing*: placement and membership shift on the order of nodes joining and models loading, not on the order of tokens generated. The hot path — a forward pass through the ring — never touches consensus.

3.2 Transport plane: one PQ wire for control and tensors

Every inter-node byte moves over ZAP. This is deliberately *one* wire for two kinds of traffic that a monolith would split across two stacks:

- **Control messages** — consensus votes, placement updates, admission requests — which Quasar already speaks over ZAP.
- **Tensor data** — the activation exchange and all-reduce that the ring performs every micro-step — which today crosses raw TCP and, under this design, crosses the same authenticated `WrapPQnet.Conn` instead.

Unifying them on one wire is the payoff of decomplexion: the ring does not need to know it is on a network of strangers, only that it holds a `net.Conn`; the transport does not need to know whether the bytes are a vote or an activation. ZAP’s zero-copy framing is what makes carrying tensors over an authenticated session affordable — there is no marshal/unmarshal tax on the activation exchange, only an AEAD seal over a buffer the ring already owns (the 64 KiB `MaxPQRecord` chunking of `conn_pq.go` maps naturally onto activation-sized writes). The identity that the handshake establishes, `PeerID`, is the same value the control plane keys membership on: the node you complete a ZAP handshake with *is* the node consensus admitted, by construction, with no separate PKI to reconcile.

3.3 Data plane: unchanged in kind

The data plane is the least-changed and that is the point. The ring’s numerics — canonical `f32` reduction, tensor/pipeline sharding, the five-backend kernels — are correct today (§2.1) and stay exactly as they are. Two wires change underneath them: the topology arrives from the control plane’s placement map rather than `RingConfig::load()`, and the `TcpStream` is a ZAP `net.Conn` rather than a bare socket. The ring code that computes does not move. This is composition over rewrite: we are not building a decentralized inference engine, we are giving the inference engine we have a control plane and an authenticated transport.

3.4 Why three planes, stated precisely

Proposition 1 (Orthogonality of the three planes). *The control, transport, and data planes can each be replaced or scaled independently without modifying the other two, because they share only two narrow, typed interfaces: the control plane emits a placement map keyed by `PeerID`, and the transport plane emits an authenticated `net.Conn` whose `PeerID` matches. The data plane consumes both and exposes neither consensus nor cryptography; the transport consumes an identity and exposes a byte stream, knowing nothing of tensors; the control consumes admission and contribution events and exposes agreed state, knowing nothing of wire framing.*

Argument. By interface inspection. The data plane’s dependencies are exactly (i) a topology — rank, world size, neighbor address, layer range — and (ii) a `net.Conn` per neighbor. Neither names a consensus type or a cipher. Swap Quasar for any state machine that produces the same placement map and the ring is unaffected; swap ZAP for any authenticated `net.Conn` with a stable peer identity and the ring is unaffected; swap the ring’s kernels (add a backend, change the quant) and neither consensus nor transport observes it. The only cross-plane invariant is that the `PeerID` the transport authenticates equals the `PeerID` the control plane admitted — a single equality, checked once at connection setup, which is precisely what `WrapPQ`’s `PeerID()` was built to support (“validator-set membership checks,” `conn_pq.go`). Three concerns, two interfaces, one shared value. □ □

The contrast with a monolithic serving stack is now sharp. In vLLM or Triton, “who is in the cluster” is a launch-time flag baked into the process, “how bytes move” is a hardcoded NCCL or gRPC channel with no peer authentication, and “what computes” is the same binary — the three concerns are one artifact, so you cannot add a permissionless member, cannot authenticate a stranger’s GPU, and cannot verify a result without forking the whole thing. Our planes are three artifacts with two seams. That is the difference between a cluster and a network.

4 Threat model and result integrity

A permissionless network inherits adversaries a private cluster never has. We state the model, the gaps in today’s substrate, and the design that closes them.

Assets and adversaries. The assets are: the integrity of an inference result (the tokens a user is billed for must be the model’s real output), the confidentiality and authenticity of activations in flight, the correctness of the placement map, and the fairness of reward. The adversaries are: a network attacker on the path between nodes (public-cloud tenants share fabric); an *intruder* who obtained a join token; and a *byzantine miner* who joined honestly but returns wrong, fabricated, or lazily-cached tensors to earn reward without doing the work.

Gap 1: the wire is plaintext ([built] problem). Today the ring is raw TCP (§2.1). A path attacker can read activations, inject a forged tensor into the all-reduce, or man-in-the-middle a neighbor. **Closure [design]:** the ZAP PQ handshake. After `WrapPQ`, every frame is AEAD-sealed and every peer is a verified ML-DSA identity; injection and eavesdropping both fail against a post-quantum authenticated channel. The tensor plane inherits the same wire security consensus already relies on.

Gap 2: the join token is a bearer secret ([built] problem). The QR token (Listing 2) is cleartext topology; whoever holds it joins (§2.2). **Closure [design]:** make the token a *handshake credential*, not the thing that grants access. Under the design the token carries the ring’s `PeerID` set (or a commitment to it); a worker presenting the token must then *complete a ZAP PQ handshake* proving possession of an admitted ML-DSA key. A photographed QR no longer joins anyone — it only names who is allowed to, and access is gated on the handshake the intruder cannot forge. The token degrades from a secret to a public address book, which is the correct security posture: identity is the credential, topology is not.

Gap 3: a miner may lie about its work (the hard one). Authentication proves *who* sent a tensor, not that the tensor is *correct*. A byzantine miner holding layers 8–15 can return plausible-looking garbage and collect reward. This is the genuinely new problem, and consensus is the tool. We propose three composable mechanisms, in increasing cost, scheduled by the control plane [design]:

1. **Threshold attestation of outputs.** A completed inference (or a placement-epoch’s batch of them) is sealed with a QuasarCert: a Photon committee of GPU nodes threshold-signs the (request, output) pair with BLS+Pulsar+ML-DSA (§2.4). A billed result carries a certificate that a supermajority attested to it, giving the user cryptographic non-repudiation and making a single lying miner insufficient to certify a bad output.
2. **Redundant re-execution / spot checks.** The scheduler occasionally assigns the same layer range to two independent miners and compares — cheap because the ring wire is already *canonical f32* (§2.1), so two honest miners on different vendors produce *bit-identical* tensors and any disagreement is a proof of fault, not a rounding artifact. The canonical-f32 property, built for cross-vendor correctness, doubles as the substrate for verifiable redundancy. A miner caught disagreeing with the honest majority is dimmed by luminance and slashed.
3. **Reputation as the economic backstop.** Luminance (§2.4) makes correct work compound and incorrect work decay: a node that fails verification multiplies its brightness by 0.9 and is sampled into fewer committees and paid for less work, converging its influence and income toward zero. Reputation is the slow, cheap layer under the fast, expensive cryptographic ones.

Byzantine fault tolerance is a property the control plane already has (Quasar tolerates $< 1/3$ adversarial stake); the contribution here is *extending it from block finality to inference finality* by threshold-signing over outputs and exploiting canonical f32 for cheap re-execution. We are explicit that this is [design]: the threshold-signing primitive is built, its application to inference outputs is proposed.

Non-goal: confidentiality from the operator. This design authenticates and verifies; it does not, by itself, hide activations from the miner that computes them. Workloads needing that require a hardware TEE tier (H100/H200 confidential computing, TDX, SEV-SNP), an orthogonal extension developed in the tenant-cloud companion [2] and out of scope here.

5 The economic layer: luminance as useful-work weighting

A permissionless network needs an incentive, and the incentive must be *useful-work*, because paying for wasted computation (proof-of-work hashing) is exactly what an inference network should not do. The luminance primitive is already a useful-work meter in miniature: it rewards nodes for correct participation and decays nodes for failure. The economic design [design] binds three quantities to it:

- **Admission by stake.** A miner joins by bonding stake in a transaction (§3.1). Stake is the sybil cost — spinning up a thousand fake GPUs is a thousand bonds — and the slashing target when verification (§4) catches a fault.
- **Reward by validated contribution.** Payment for an inference accrues to the miners whose outputs were attested, weighted by luminance and by measured work (tokens $\times$ model size $\times$ layer share). Because luminance rises with correct work and falls with incorrect work, expected income tracks *honest throughput*, and a lazy or lying miner earns less than its electricity costs.

- **Routing by brightness.** The Photon committee that ratifies placement and verifies results draws with luminance weight, so brighter (more reliably correct) nodes are preferentially given work — reputation and revenue reinforce, which is what makes “contribute a GPU, get paid” a stable equilibrium rather than a race to the bottom.

The mechanism reuses one existing primitive (luminance) for reputation, one existing primitive (stake-weighted BFT) for sybil-resistance and slashing, and one existing primitive (Photon) for work routing. The realized system is “any dev, any miner, contributes a GPU and is paid for inference work,” with the payment denominated in the network’s native token and the fairness enforced by the same consensus that finalizes the placement map. This is the point of contact with the sovereign-cloud economics of the tenant-cloud companion [2], which develops the wallet, staking, and settlement primitives this layer would sit on.

6 Migration path: from LAN appliance to public network

The design is only credible if there is a continuous path from what runs today to the network, with no flag day. There is, in four stages, each shippable and independently valuable.

Stage 0 (today, [built]). Static RING_CONFIG JSON, QR join token, raw-TCP ring, canonical f32 across vendors. Works on a trusted LAN; this is the baseline.

Stage 1 — authenticate the wire ([design], smallest step). Replace the ring’s `TcpStream` with a `ZAP WrapPQ net.Conn` and make the join token gate on a PQ handshake (§4, Gaps 1–2). No consensus yet: the topology is still the static map, but the wire is now authenticated and encrypted and the token is a credential. This alone converts the appliance into something safe to run across untrusted networks, and it exercises the engine’s ZAP module end-to-end — turning the [partial] `zap://` transport on (§2.3) is precisely this step.

Stage 2 — consensus owns the map ([design]). Stand up a Quasar chain whose state is the membership and placement map. Nodes read topology from the chain instead of `RingConfig::load()`; `hanzo cluster join` becomes an admission transaction rather than a local launch. Placement is still operator-proposed and consensus-ratified (permissioned membership, BFT config). The ring code is untouched; only the source of its topology moves from file to chain.

Stage 3 — permissionless miners and paid work ([design]). Open admission to staked strangers, bind luminance to validated inference contribution, and turn on result attestation and spot-check re-execution (§4–§5). This is the full network: anyone bonds, joins, is placed by consensus, serves layers over authenticated ZAP, is verified by a Photon committee, and is paid by luminance-weighted contribution.

The staging matters because each stage stands alone. Stage 1 is worth shipping even if Stages 2–3 never do (it fixes a real security gap). Stage 2 is worth shipping for a single operator’s multi-datacenter fleet (BFT config, no static files) without any economics. Only Stage 3 needs the token and slashing machinery. There is no rewrite between stages — the same ring, the same ZAP, more of the control plane each time.

7 Connection to the latency-physics companion

A pipeline-parallel ring over a wide-area network has a latency budget a single box does not: each pipeline stage adds a network hop to the critical path of every token. The companion latency-physics analysis [3] argues that this budget is spendable precisely because two techniques compose over ZAP. First, **pipeline parallelism hides transport latency under compute**: with enough micro-batches in flight, a stage’s forward pass overlaps the next stage’s activation transfer, so the ring’s throughput approaches the slowest stage’s compute rate rather than the sum of hop latencies — and ZAP’s zero-copy framing minimizes the per-hop transport term to an AEAD seal, not a marshal. Second, **speculative decoding amortizes the round-trip**: a draft model proposes several tokens that the ring verifies in one pipelined pass, so the wide-area round-trip is paid once per *accepted block* of tokens instead of once per token. The engine already factors speculative decoding behind one `SpeculativeProposer` trait [1]; over a consensus-managed ZAP ring, the draft can even run on a nearby, brighter node while the target pipeline spans the fleet. The three-plane fabric is what makes these composable: the data plane does the pipelining and speculation, the transport plane makes each hop cheap and authenticated, and the control plane places draft and target where the latency budget is best — decomplexed, so each optimization lives in exactly one plane. We cite the companion for the quantitative budget; here it is enough that the architecture does not fight the physics.

8 Status: built, partial, and designed

Table 1 is the honest ledger. It is the table to read first: it says exactly which sentences of this paper are code on `main` and which are blueprint. The pattern is deliberate — every *primitive* is built and cited; every *composition* into the network is designed. That is what makes this a buildable blueprint rather than a wish.

Reproducing the built numbers. The ZAP figures (2.9 ns parse, $17\times/345\times$ versus JSON-RPC, 114K/376K TPS) reproduce with `go test -bench=. -benchmem` in `luxfi/zap` and `luxfi/consensus/bench/`; the Photon/Wave/luminance costs (3.03 μ s, 3.38 μ s, 72 ns) reproduce with `GOWORK=off go test -bench=. ./bench/` in `luxfi/consensus`, measured on Apple M1 Max. The cross-vendor ring runs with a `RING_CONFIG` JSON per rank and `hanzo cluster init/join`; the cross-backend engine numbers are reproduced by the protocol in the companion stack paper [1]. None of these numbers is produced by this paper; each is attributed to its source so the blueprint’s foundation is auditable.

9 Limitations

The rigor is the persuasion, so the costs are as plain as the benefits.

- **The network is a composition, not a shipped network.** The primitives are built and cited; the assembly of §3–§5 is [design]. Nothing here should be read as “Hanzo runs a permissionless GPU network today.” It runs a cross-vendor ring on a trusted mesh (Stage 0) and ships every primitive the network needs.
- **The ZAP engine wire is not yet turned on.** The Rust ZAP module is [partial]: its in-process surface is exercised, but the `zap://` out-of-process transport is a documented stub and

a few methods return “not yet wired” (§2.3). Stage 1 depends on completing it. The Go ZAP wire it mirrors is production-deployed, so the risk is integration, not invention.

- **Wide-area tensor transport is unproven at scale.** The ring was built for a LAN. Pipeline parallelism over a wide-area network is argued to be latency-tolerant (§7) but not measured here; the companion latency paper carries that budget, and a real deployment must validate it. Bandwidth, not just latency, may bound activation exchange for large hidden dimensions.
- **Result verification adds cost.** Threshold attestation and redundant re-execution (§4) are not free: a QuasarCert verifies in 2–5 ms and spot-check redundancy duplicates work on sampled requests. The scheme is designed to make the *expected* overhead small (attest per epoch, spot-check a fraction) but it is a real tax the monolithic single-trust model does not pay — the price of permissionlessness.
- **Confidentiality from the operator is out of scope.** This design authenticates and verifies; it does not hide activations from the computing miner without a TEE tier (§4). Sensitive workloads need that orthogonal extension.
- **Luminance re-binding is a policy change with game-theoretic edges.** Moving the *Illuminate* trigger from vote-success to inference-contribution is a one-line mechanism change but a large incentive-design question: collusion among a committee, grinding on the verification lottery, and reward-share disputes across a pipeline are real and not solved by the primitive alone. We name them; we do not claim to have priced them.

10 Conclusion

The heterogeneous GPU ring already solves the hard, unglamorous problem — one model computing correctly across ROCm, CUDA, and Metal, bit-for-bit, over a canonical wire. What it lacks to become a network is not more numerics but two planes it does not have: a control plane to decide membership and placement among parties who do not trust each other, and a transport plane to authenticate the strangers it talks to. Both already exist as best-in-class Lux primitives — Quasar consensus and the ZAP post-quantum wire — and the entire contribution of this paper is to hold the three concerns apart and compose them: consensus schedules, ZAP carries, the ring computes. That decomplexion is what a monolith cannot do and what makes “any developer, any miner, contributes a GPU and is paid for inference work” a buildable system rather than a slogan. We have drawn the line between built and designed down the middle of every claim: the ring, the scan-to-join CLI, the ZAP wire with its verified *PeerID*, the Quasar engine with luminance and threshold signing are code on *main*, cited and measured; the network that composes them — consensus-owned membership, the ZAP-transported ring, permissionless admission, and result verification by threshold-signing over outputs — is design, staged for a migration with no flag day. The primitives are done. The composition is the blueprint. We publish it so the rest of the AI-infrastructure community can build on the same three planes.

References

- [1] Hanzo AI — ML Systems. *Hanzo ML and Hanzo Engine: A Unified Multi-Backend, Multi-Modal Inference Stack*. Hanzo AI white paper, 2026 (companion; the data-plane engine, cross-vendor ring, and five-backend decode parity).

- [2] Hanzo AI Research. *The Unified Sovereign-Tenant Cloud: Linear Shared-Nothing Scaling via Per-Tenant SQLite and In-Process Composition*. Hanzo Industries white paper, 2026 (companion; the `hanzo.network` fabric, wallet/staking, TEE trust tiers, and PQ identity primitives).
- [3] Hanzo AI — ML Systems. *The Latency Physics of Decentralized Decode: Pipeline Parallelism and Speculative Decoding over ZAP*. Hanzo AI white paper, 2026 (companion; the wide-area latency budget referenced in §7).
- [4] Lux Industries. *Quasar: Post-Quantum DAG-Native BFT Consensus* (LP-020 / LP-105). Lux Network, 2026. <https://github.com/luxfi/consensus>.
- [5] Lux Industries. *luxfi/consensus*: the Quasar engine — `protocol/photon` (Fisher-Yates committee selection, luminance), `protocol/wave` (FPC threshold voting), `protocol/focus`, `protocol/quasar` (BLS + Pulsar + ML-DSA threshold signing). Source and benchmarks (`bench/`, `README.md`, `LLM.md`). <https://github.com/luxfi/consensus>.
- [6] Lux Industries. *ZAP: Zero-copy Application Protocol* — the universal wire. github.com/luxfi/zap (`zap.go`, `conn_pq.go` PQ handshake / `WrapPQ` / `PeerID`, `README.md` benchmarks).
- [7] Lux Industries. *LP-201: ZAP as the Universal Wire* and *LP-200: The ZAP Stack*. Lux Proposals, 2026. <https://github.com/luxfi/lps> (the $17\times/345\times$ vs JSON-RPC and $\sim 4.5\times$ vs gRPC measurements).
- [8] National Institute of Standards and Technology. *FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)*. NIST, 2024.
- [9] National Institute of Standards and Technology. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)*. NIST, 2024.
- [10] D. Boneh, B. Lynn, and H. Shacham. Short Signatures from the Weil Pairing. *Journal of Cryptology*, 17(4):297–319, 2004.
- [11] A. Borzunov et al. *Petals: Collaborative Inference and Fine-tuning of Large Models*. ACL System Demonstrations, 2023 (decentralized inference over volunteer GPUs; no BFT control plane or PQ-authenticated transport).
- [12] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. *SOSP*, 2023 (the monolithic single-operator serving baseline).
- [13] B. Pinkas and M. Reiter, and the committee-grinding threat that motivates rejection-sampled uniform selection in `photon/emitter.go`. (Bias-free sortition under a grinding adversary.)

Component	Plane	Status	Evidence / source
Cross-vendor inference ring (TP/PP, canonical f32)	Data	[built]	distributed.rs, distributed/socket.rs, ops.rs [1]
NCCL + portable ring backends	Data	[built]	hanzo-quant/src/distributed/
hanzo cluster scan-to-join (bs58/QR token)	Data	[built]	hanzo-cli/src/cluster.rs
Five-backend engine, decode parity, quant zoo	Data	[built]	companion stack paper [1]
ZAP Go wire (zero-copy, 17×/345× vs JSON-RPC)	Transport	[built]	luxfi/zap [6, 7]
ZAP PQ handshake, WrapPQ, verified PeerID	Transport	[built]	conn.pq.go
ZAP engine module (in-process adapter surface)	Transport	[partial]	engine/src/zap/*.rs (server/- client)
out-of-process zap:// (capnp wire)	Transport	[partial]	engine/src/zap/transport.rs (stub)
Quasar BFT engine; Photon, Wave, Focus	Control	[built]	luxfi/consensus [4, 5]
Luminance reputation (100 base, ×1.1/×0.9)	Control	[built]	photon/luminance.go
BLS+Pulsar+ML-DSA thresh- old signing (QuasarCert)	Control	[built]	protocol/quasar [5]
Ring transport over ZAP (retire raw TCP)	Transport	[design]	§3.2, Stage 1
Join token as PQ handshake cre- dential	Transport	[design]	§4, Gap 2
Consensus-owned membership & placement map	Control	[design]	§3.1, Stage 2
Permissionless miner admission (stake bond)	Control	[design]	§5, Stage 3
Luminance ← validated infer- ence contribution	Control	[design]	§3.1
Result verification (threshold attest + spot check)	Control	[design]	§4, Gap 3
Luminance-weighted reward / routing	Control	[design]	§5

Table 1: Built-versus-designed ledger. Every primitive (upper block) is code on `main`, cited to its file with its measured cost where relevant; every network composition (lower block) is design, cited to the section that specifies it and the migration stage that ships it.